Python OOP Basics

1. Classes and Objects
A class is a blueprint for creating objects. An object is an instance of a class
.
You define a class using the "class" keyword. Example:

```
class Dog:
    def __init__(self, name):
        self.name = name

    def bark(self):
        return f"{self.name} says Woof!"

my_dog = Dog("Rex")
print(my_dog.bark())
```

Here, Dog is the class, and my_dog is an object (instance) of that class.

2. The __init__ Method
__init__ is a special method called a constructor. It runs automatically when a
new
object is created, and it is used to set up initial attribute values. The first
parameter of every instance method, including __init__, is "self", which refers
to
the specific object being created or acted on.

3. Attributes and Methods
Attributes are variables that belong to an object (like self.name above). Method
s
are functions defined inside a class that describe the behaviors of an object (l
ike
bark()). Attributes store data; methods define actions.

4. Inheritance
Inheritance lets a class (child) reuse and extend the behavior of another class
(parent). Example:

```
class Puppy(Dog):
    def play(self):
        return f"{self.name} is playing!"
```

Puppy inherits the bark() method from Dog and adds its own play() method. This
avoids rewriting shared code and models an "is-a" relationship (a Puppy is a Dog
).

5. Encapsulation
Encapsulation means bundling data and the methods that operate on that data insi
de
one unit (the class), and restricting direct access to some of an object's
components. In Python, a leading underscore (e.g. _balance) signals "internal us
e
only" by convention, though it is not strictly enforced by the language.

6. Polymorphism
Polymorphism means different classes can define methods with the same name, and
the correct method is chosen automatically based on the object's actual class.
For example, both Dog and Cat classes could each define a speak() method, and
calling speak() on either object produces the right sound without the caller
needing to know which exact class it is.

7. Class Variables vs Instance Variables

A class variable is shared across all instances of a class and is defined directly
inside the class body. An instance variable is unique to each object and is usually defined inside __init__ using self. Class variables are useful for values that should stay the same for every object, like a species name; instance variables hold data that differs per object, like a name or age.